Unité 11 — Cahier de laboratoire (étudiant)

Livrable C — Fuzzing AFL++ : campagne, triage, correctif et harnais | UE « Software Security »

> Cadre : environnement Docker isolé, sur les cibles pédagogiques fournies. Le code C du Lab 11.4 est fourni intégralement : l'objectif est de comprendre le harnais, l'exécuter et interpréter ses résultats, pas de programmer en C de façon autonome.

Lab 11.0 — Préparer l'environnement

Depuis le dossier U11-fuzzing/labs/lab01-afl sur la VM, lancer le conteneur. Si votre configuration Compose utilise LAB_UID/LAB_GID, les définir avant le lancement :

```
export LAB_UID=$(id -u)
export LAB_GID=$(id -g)
docker compose build
docker compose run --rm lab bash
```

Dans le conteneur, vérifier les outils puis compiler les quatre binaires de référence :

```
cd /workspace
afl-fuzz -V
clang --version
./scripts/build.sh
ls -l build/vuln_fuzz build/vuln_asan \
      build/vuln_fixed_fuzz build/vuln_fixed_asan
```

> Important : ne pas modifier /proc/sys/kernel/core_pattern dans le conteneur. Il est en lecture seule dans notre environnement. Les campagnes AFL++ utilisent AFL_SKIP_CPUFREQ=1 et AFL_I_DONT_CARE_ABOUT_MISSING_CRASHES=1. Cette dernière option contourne le contrôle AFL++ mais peut affecter la classification de certains crashs et timeouts. Confirmer les crashs avec ASan.

Lab 11.1 — Première campagne AFL++

Objectifs : observer le fuzzing guidé par couverture ; identifier un crash et lire les statistiques.

Le programme src/vuln.c reconnaît l'en-tête FUZZ puis copie le reste dans un tampon de pile de 64 octets sans contrôle de longueur. Le corpus initial est fourni dans corpus/.

```
AFL_SKIP_CPUFREQ=1 AFL_I_DONT_CARE_ABOUT_MISSING_CRASHES=1 \
  afl-fuzz -i corpus -o findings -- ./build/vuln_fuzz
```

Laisser tourner environ une à deux minutes, observer exec speed, corpus count, stability et saved crashes, puis interrompre avec Ctrl+C. Les temps et nombres varient selon la machine.

```
cat findings/default/fuzzer_stats | grep -E \
 'execs_done|execs_per_sec|corpus_count|stability|saved_crashes'
find findings/default/crashes -maxdepth 1 -type f -name 'id:*'
```

Questions : pourquoi commencer avec une graine contenant « FUZZ » ? Que signifie la stabilité ? Pourquoi le nombre de crashs peut-il varier d'une exécution à l'autre ?

Lab 11.2 — Reproduire et minimiser le crash

Objectifs : identifier la cause avec AddressSanitizer (ASan) ; réduire la taille de l'entrée déclenchant le bug.

```
CRASH=$(find findings/default/crashes -maxdepth 1 \
  -type f -name 'id:*' | sort | head -n 1)
test -n "$CRASH" || { echo "Aucun crash sauvegarde"; exit 1; }
printf '%s\n' "$CRASH"
./build/vuln_asan "$CRASH"
```

Relever le type d'erreur, le fichier, la ligne et le nombre d'octets écrits. Puis minimiser le crash :

```
AFL_SKIP_CPUFREQ=1 AFL_I_DONT_CARE_ABOUT_MISSING_CRASHES=1 \
  afl-tmin -i "$CRASH" -o crash_min.bin -- ./build/vuln_fuzz
wc -c "$CRASH" crash_min.bin
./build/vuln_asan crash_min.bin
```

Questions : quelle est la taille du tampon ? Pourquoi minimiser un fichier de crash ? Le crash minimisé déclenche-t-il encore une alerte ASan ?

Lab 11.3 — Vérifier le correctif et re-fuzzer

Objectifs : comparer une copie non bornée et une copie bornée ; vérifier la non-régression sur les entrées testées.

Dans src/vuln_fixed.c, la longueur copiée est limitée à 63 octets afin de réserver un octet au terminateur nul :

```
size_t paylen = len - 4;
if (paylen > sizeof(buf) - 1)
    paylen = sizeof(buf) - 1;
memcpy(buf, data + 4, paylen);
buf[paylen] = '\0';
```

```
./build/vuln_fixed_asan crash_min.bin
AFL_SKIP_CPUFREQ=1 AFL_I_DONT_CARE_ABOUT_MISSING_CRASHES=1 \
  afl-fuzz -i corpus -o findings2 -- ./build/vuln_fixed_fuzz
```

Laisser tourner environ une minute. Comparer le nombre de crashs sauvegardés avec celui du Lab 11.1. Attention : zéro crash pendant une campagne ne prouve pas l'absence de toute vulnérabilité.

Lab 11.4 — Comprendre un harnais et valider sa correction

Objectifs : comprendre le rôle d'un harnais qui lit sur l'entrée standard (stdin) ; observer un débordement ; confirmer son correctif. Le code est fourni : aucune rédaction autonome en C n'est demandée.

Code vulnérable fourni : src/harness/harness.c. Il ne traite que les messages commençant par TEST et copie ensuite leur contenu dans un tampon de 16 octets.

```c
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

void analyse_message(const unsigned char *data, size_t size) {
    char buffer[16];
    if (size < 4) return;
    if (memcmp(data, "TEST", 4) != 0) return;
    /* Vulnerabilite volontaire : copie sans controle */
    memcpy(buffer, data + 4, size - 4);
    puts("Message analyse");
}
int main(void) {
    unsigned char input[1024];
    size_t n = fread(input, 1, sizeof(input), stdin);
    analyse_message(input, n);
    return 0;
}
```

Étape A : compiler le harnais et créer une graine valide :

```
mkdir -p build corpus_harness src/harness
afl-clang-fast -O0 -g -fno-stack-protector \
  -o build/harness_fuzz src/harness/harness.c
clang -O0 -g -fsanitize=address -fno-omit-frame-pointer \
  -o build/harness_asan src/harness/harness.c
printf 'TESThello' > corpus_harness/seed1
```

Étape B : lancer la campagne et rejouer un crash sur le binaire ASan :

```
AFL_SKIP_CPUFREQ=1 AFL_I_DONT_CARE_ABOUT_MISSING_CRASHES=1 \
  afl-fuzz -i corpus_harness -o findings_harness -- ./build/harness_fuzz
# Apres Ctrl+C :
CRASH_HARNESS=$(find findings_harness/default/crashes \
  -maxdepth 1 -type f -name 'id:*' | sort | head -n 1)
./build/harness_asan < "$CRASH_HARNESS"
```

Étape C : utiliser le correctif fourni src/harness/harness_fixed.c. Il limite la copie à la taille du tampon, puis exécuter :

```
clang -O0 -g -fsanitize=address -fno-omit-frame-pointer \
  -o build/harness_fixed_asan src/harness/harness_fixed.c
afl-clang-fast -O0 -g \
  -o build/harness_fixed_fuzz src/harness/harness_fixed.c
./build/harness_fixed_asan < "$CRASH_HARNESS"
echo "Code retour : $?"
AFL_SKIP_CPUFREQ=1 AFL_I_DONT_CARE_ABOUT_MISSING_CRASHES=1 \
  afl-fuzz -i corpus_harness -o findings_harness_fixed \
  -- ./build/harness_fixed_fuzz
```

Le correctif fourni doit appliquer la borne suivante avant la copie :

```
size_t longueur = size - 4;
if (longueur > sizeof(buffer))
    longueur = sizeof(buffer);
memcpy(buffer, data + 4, longueur);
```

Compte rendu (court) : relever le nombre d’exécutions, les crashs sauvegardés, la ligne ASan responsable, la taille du tampon, la taille de l’écriture et le résultat du re-fuzzing. Expliquer en deux phrases pourquoi le correctif empêche le débordement observé.

Questions : à quoi sert le harnais ? Pourquoi utiliser < "$CRASH_HARNESS" plutôt qu’un argument de fichier ? Pourquoi zéro crash ne constitue-t-il pas une preuve formelle de sécurité ?

Livrables et durée indicative

Un court rapport pour chaque lab (captures ou extraits de statistiques, commandes et réponses). Durées indicatives : 11.1 = 1 h ; 11.2 = 1 h ; 11.3 = 1 h ; 11.4 = 2 h. Les campagnes peuvent s’exécuter pendant les explications.